Clase 5

@September 9, 2026

Introducción

Hoy concluimos la inicialización del motor gráfico con la creación del **Vertex Buffer** en GPU e implementamos la lógica del ciclo de renderizado en el método `Engine::Render()`. También, realizamos la integración formal de la arquitectura de la API con la aplicación executable (`Sandbox/main.cpp`).

Desarrollo

Vertex Buffer

Para enviar la geometría declarada en CPU hacia la memoria de la GPU (VRAM), se configuraron las estructuras de DirectX 11:

- `D3D11_BUFFER_DESC`: Describe las propiedades del espacio en memoria de la GPU:
  - `ByteWidth`: Tamaño total en bytes, calculado mediante `sizeof(vertices)`.
  - `Usage = D3D11_USAGE_IMMUTABLE`: Indica que el búfer es de solo lectura para la GPU y sus datos no cambiarán desde la CPU después de la creación.
  - `BindFlags = D3D11_BIND_VERTEX_BUFFER`: Especifica que el búfer se enlazará como un búfer de vértices en la etapa *Input Assembler*.
- `D3D11_SUBRESOURCE_DATA`: Contiene el puntero a los datos iniciales en RAM (`pSysMem = vertices`).
- **Creación:** Se ejecuta `device->CreateBuffer(...)`. Si la llamada falla (`FAILED(result)`), se liberan los recursos creados con `ReleaseResources()` y se aborta la inicialización.

```
D3D11_BUFFER_DESC vertexBufferDescription{};
vertexBufferDescription.ByteWidth = static_cast<UINT>(sizeo
f(vertices));
vertexBufferDescription.Usage = D3D11_USAGE_IMMUTABLE;
```

```
vertexBufferDescription.BindFlags = D3D11_BIND_VERTEX_BUFFE
R;

D3D11_SUBRESOURCE_DATA initialVertexData{};
initialVertexData.pSysMem = vertices;

result = engine.device->CreateBuffer(
    &vertexBufferDescription,
    &initialVertexData,
    &engine.vertexBuffer
);
```

Engine::Render()

El método `Engine::Render()` se encarga del dibujo cada frame:

Verificación de Seguridad:

Confirma que todos los objetos de DirectX (contexto, *swap chain*, shaders, etc.) estén instanciados antes de ejecutar llamadas a la GPU.

```
if (!engine.context ||
    !engine.swapChain ||
    !engine.renderTarget ||
    !engine.vertexBuffer ||
    !engine.inputLayout ||
    !engine.vertexShader ||
    !engine.pixelShader)
{
    return;
}
```

Asignación del Target y Limpieza del Backbuffer:

Se asigna la vista de renderizado mediante `OMSetRenderTargets`

Se define un color de fondo azul marino `clearColor[] = { 0.03f, 0.04f, 0.08f, 1.0f }`

Se llama a `ClearRenderTargetView` para borrar la imagen del fotograma anterior. Sin esta limpieza, los fotogramas se superpondrían en memoria generando un

efecto de arrastre visual (*artefacto similar a las ventanas superpuestas cuando se congela el sistema*).

```
constexpr float clearColor[]
{
    0.03f,
    0.04f,
    0.08f,
    1.0f
};

engine.context->OMSetRenderTargets(
    1,
    &engine.renderTarget,
    nullptr
);

engine.context->ClearRenderTargetView(
    engine.renderTarget,
    clearColor
);
```

Configuración de Etapas del Pipeline:

`IASetVertexBuffers`: Enlaza el búfer de vértices definiendo el paso (*stride* = `sizeof(Vertex)`) y el desplazamiento (*offset* = `0`).

`IASetInputLayout`: Aplica el formato de entrada de los vértices.

`IASetPrimitiveTopology`: Establece `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST` para interpretar cada grupo de 3 vértices como un triángulo independiente.

`VSSetShader` / `PSSetShader`: Activa el *Vertex Shader* y el *Pixel Shader* en la GPU.

```
engine.context->IASetVertexBuffers(
    0,
    1,
    &engine.vertexBuffer,
    &stride,
    &offset
);
```

```
engine.context->IASetInputLayout(
    engine.inputLayout
);

engine.context->IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST
);

engine.context->VSSetShader(
    engine.vertexShader,
    nullptr,
    0
);

engine.context->PSSetShader(
    engine.pixelShader,
    nullptr,
    0
);
```

Dibujado y Presentación:

`engine.context->Draw(3, 0)`: Envía a procesar 3 vértices desde el índice 0.

`engine.swapChain->Present(1, 0)`: Intercambia el *BackBuffer* con el *FrontBuffer* sincronizando el renderizado con la tasa de refresco del monitor (V-Sync).

Engine::Shutdown

Se implementó el método `Shutdown()` para ejecutar una salida limpia del motor invocando `ReleaseResources()` cuando la aplicación finaliza.

```
void Engine::Shutdown() noexcept
{
    if (m_implementation)
        m_implementation->ReleaseResources();
}
```

Integración de la Sandbox (`Sandbox/main.cpp`)

Procedimiento de Ventana (`WindowProcedure`): Manipula los mensajes enviadas por el sistema operativo:

- `WM_CLOSE` : Destruye la ventana (`DestroyWindow`).
- `WM_DESTROY` : Envía la orden de término al hilo del sistema (`PostQuitMessage(0)`).
- `WM_ERASEBKGND` : Retorna `1` explícitamente para evitar que Windows borre el fondo con el color blanco predeterminado del sistema, evitando parpadeos ya que DirectX limpia todo el lienzo.

```
LRESULT CALLBACK
WindowProcedure(HWND window, UINT message, WPARAM wParam, L
PARAM lParam) {
    switch (message)
    {
    case WM_CLOSE:
        DestroyWindow(window);
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;

    case WM_ERASEBKGND:
        //DirectX limpia y dibuja toda la ventana
        return 1;

    default:
        return DefWindowProcW(
            window,
            message,
            wParam,
            lParam
        );
    }
}
```

Ajuste del Área Cliente (`AdjustWindowRect`): Modifica el tamaño de la ventana (`1280x720`) para asegurar que las dimensiones especificadas correspondan al área útil dentro de los bordes y barra de título de la ventana.

Bucle Principal No Bloqueante (`PeekMessageW`):

- Se sustituyó `GetMessageW` por `PeekMessageW(..., PM_REMOVE)` para consultar la cola de mensajes sin detener la ejecución.
- Si no hay mensajes pendientes del sistema operativo, el bucle llama continuamente a `engine.Render()` para generar fotogramas en tiempo real.
- **Optimización de Pausa:** Se incluyó la comprobación `IsIconic(window)`. Si la ventana se minimiza, se llama a `WaitMessage()` para suspender el ciclo de renderizado y ahorrar uso de CPU/GPU hasta que la ventana vuelva a estar activa.

```
while (running)
{
    while (PeekMessageW(
        &message,
        nullptr,
        0,
        0,
        PM_REMOVE))
    {
        if (message.message == WM_QUIT)
        {
            running = false;
            break;
        }

        TranslateMessage(&message);
        DispatchMessageW(&message);
    }

    if (!running)
        break;

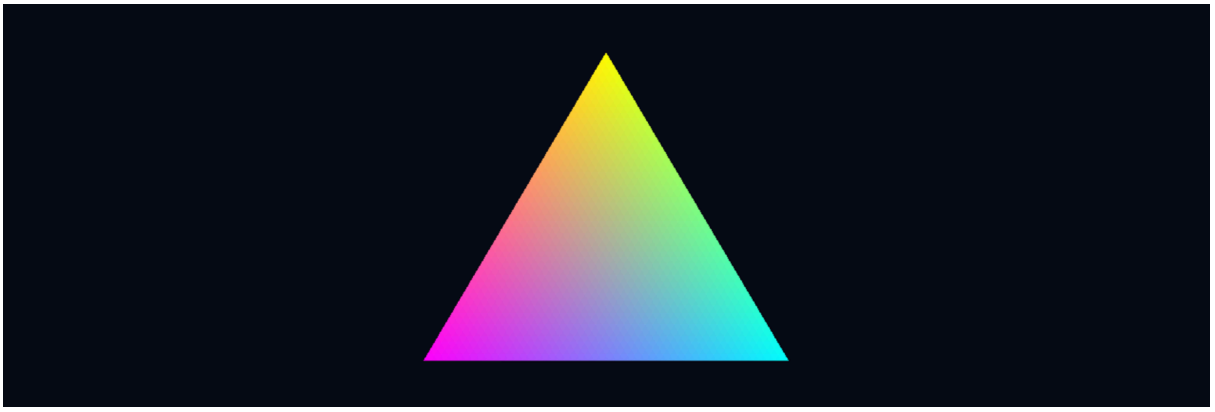
    if (IsIconic(window))
    {
```

```
            WaitMessage();
            continue;
        }

        engine.Render();
    }
```

Resultado Visual

Al compilar y ejecutar el proyecto, se despliega una ventana Win32 de 1280×720 píxeles mostrando el triángulo interpolado mediante *Shaders* sobre el fondo azul oscuro.



Conclusión

Que bueno, siempre es satisfactorio cuando todo funciona justo como el profe dice. Noté que aunque fue súper básico, ya fui capaz de resolver yo solito sin la IA los problemas que me aparecían a partir de los mensajes de error de compilación y de build. Ya me estoy familiarizando con el ambiente.

Glosario

`D3D11_USAGE_IMMUTABLE`

Indicador de uso de recursos en DirectX 11 que especifica que un búfer será escrito una única vez durante su creación y la CPU no volverá a modificarlo en el tiempo de ejecución.

`OMSetRenderTargets` **(Output Merger Stage)**

Función del contexto que enlaza uno o más destinos de renderizado (*Render Targets*) y el búfer de profundidad a la última etapa del *pipeline* de renderizado.

`D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST`

Modificador de la etapa *Input Assembler* que le indica a la GPU cómo interpretar la secuencia de vértices, agrupando cada 3 vértices consecutivos como las esquinas de un triángulo.

`WM_ERASEBKGND`

Mensaje de Win32 enviado a una ventana cuando su fondo necesita ser redibujado. Retornar `1` le indica a Windows que la aplicación ya procesó la limpieza del fondo manualmente.

`PeekMessageW` **vs** `GetMessageW`

`GetMessageW` detiene la ejecución del programa en espera de un evento de Windows, mientras que `PeekMessageW` verifica si hay mensajes en la cola y regresa el control inmediatamente, siendo ideal para motores de renderizado interactivos (*Game Loops*).

`IsIconic`

Función de la API de Windows que comprueba si una ventana especificada se encuentra en estado minimizado.

`Present(1, 0)`

Método del *SwapChain* encargado de enviar el contenido del búfer posterior a la pantalla. El parámetro `1` activa la Sincronización Vertical (V-Sync).